Orbis: Guiding Symbolic Execution Techniques to Maximize Option-Related Branch Coverage (Artifact)

1 PAPER TITLE

Orbis: Guiding Symbolic Execution Techniques to Maximize Option-Related Branch Coverage

2 LINK TO THE ACCEPTED PAPER

The accepted paper is included in the artifact as "`ASE_26_Orbis_Paper.pdf`". It is also available at: https://github.com/minjongkim99/orbis/blob/main/ASE_26_Orbis_Paper.pdf

3 PURPOSE

This artifact provides the implementation, execution environment, benchmark configurations, target-specific data, and experimental results for Orbis.

Orbis guides symbolic execution toward branches affected by command-line options. During testing, it constructs option arguments, selects promising test-cases as seeds, invokes symbolic executors such as KLEE, and records the accumulated branch coverage after each iteration.
The artifact contains:

- the Orbis implementation;
- a pre-built Docker image and a Dockerfile;
- benchmark-building and experiment scripts;
- option dictionaries, option-related branches, and path constraints;
- a tracer for preparing additional target programs;
- scripts for reporting branch coverage and detected bugs; and
- archived experimental outputs and coverage logs.

A smoke test using `grep-3.4` is provided to verify that Orbis executes correctly and produces coverage results. Instructions for the full experiment are also provided.

4 BADGE

We apply for the following three badges.

- **Artifacts Available:** The artifact is publicly accessible through GitHub and permanently archived on Zenodo under the DOI: https://doi.org/10.5281/zenodo.21767392. The archived artifact includes the Orbis source code, benchmark configurations, target-specific data, experimental results, Dockerfile, and documentation required to access and use the artifact.
- **Artifacts Functional:** The artifact provides a complete and executable environment for running Orbis and validating its main functionality. A pre-built Docker image is provided with Orbis and the dependencies required for the smoke test. The artifact also includes benchmark programs, option dictionaries, option-related branch data, and scripts for reporting coverage and detected bugs. By following the instructions in `README.md`, reviewers can run Orbis, generate test cases, and inspect the resulting coverage data.
- **Artifacts Reusable:** The artifact is designed to support reuse and extension by other researchers. Its implementation is organized into separate components for Orbis, benchmark construction, tracing, target-specific data, and result analysis. The included tracer can be used to generate option dictionaries and option-related branch information for additional configurable command-line programs. The artifact also provides benchmark-building scripts, experiment configurations, and documentation for applying Orbis beyond the programs evaluated in the paper.

5 TECHNOLOGY

The tested environment and requirements are:

- operating system: x86-64 Linux;
- architecture: `linux/amd64`;
- Docker (tested with Docker 24.x);
- minimum memory: 8 GB RAM;
- recommended memory: 16 GB RAM or more;
- storage: at least 20 GB of free disk space; and
- Internet access for downloading the Docker image.

The Docker image contains Orbis, Python, LLVM, KLEE, STP, and the benchmark required for the smoke test. No additional installation is required to run Orbis. The Docker image took approximately two minutes to download in our test environment. The smoke test uses a 360-second symbolic-execution budget is expected to complete in approximately 10 minutes and within 30 minutes in the tested environment.

6 PROVENANCE

The permanent archival copy is available on **Zenodo**[1]. The development repository is available at **minjongkim99/orbis**[2]. The pre-built Docker image is available from **Docker Hub**[3].

7 INSTRUCTIONS

Pull the pre-built Docker image:

```
$ docker pull minjongkim99/orbis-ase26:v1.3
```

Start the container:

```
$ docker run --rm -it --ulimit stack=-1:-1 minjongkim99/orbis-ase26:v1.3 /bin/bash
```

Run the six-minute smoke test from the benchmark directory:

```
$ orbis -p grep -t 360 -d Orbis_TEST grep-3.4/obj-llvm/src/grep.bc grep-3.4/obj-gcov/src/grep
```

During execution, Orbis reports the iteration number, selected option argument, elapsed time, and accumulated coverage. Coverage information is stored in "`<output_dir>/coverage.csv`".
After execution, to generate a graphical coverage report:

```
$ python3 report_coverage.py --benchmark grep-3.4 Orbis_TEST
```

It also reports the accumulated coverage results for the input directories as the following table.

```
| output_dir | coverage |
| ---------- | -------- |
| Orbis_TEST | 2662 |
```

To inspect detected bugs:

```
$ python3 report_bugs.py --benchmark grep-3.4 Orbis_TEST
$ cat bug_result.txt
```

Detailed instructions for the smoke test, full experiments, result files, repository structure, and preparation of additional programs are provided in `README.md`.

[1] https://doi.org/10.5281/zenodo.21767392
[2] https://github.com/minjongkim99/orbis
[3] https://hub.docker.com/repository/docker/minjongkim99/orbis-ase26/general